Shufflerless-CI

RoPE-Safe Latent–Residual KV for Stationary Collaborative Decoding

No large tensor crosses the CPU–GPU (or device) boundary
during steady-state decoding

Shufflerless-CI converts the memory wall from an unpredictable runtime swapping problem into an explicitly budgeted representation-and-placement problem. A fixed, RoPE-safe projection creates the low-rank KV bulk; only small input-dependent innovations are compressed online. The cache remains stationary and attention never materializes the full old KV.

This July 2026 revision separates *what* is compressed from *when*: a weight-side, offline latent projection handles the stable bulk, while a block-wise online sparse residual preserves prompt-specific outliers. It then co-optimizes that representation with the layer partition and memory reserve on the FSU poster's cluster.

Research proposal • July 22, 2026

Contents

1 Why this idea, among the six **2**

2 Motivation **2**
2.1 The three failure modes of compression-alone 2

3 Background and related work **3**
3.1 The FSU pipelined-inference poster (the base system) 3
3.2 KV-cache compression — and the novelty boundary 3
3.3 Collaborative / distributed attention (the aggregation primitive) 3
3.4 Tensor foundations 4

4 Method **4**
4.1 The stationarity principle and per-stage memory layout 4
4.2 Component 1 — fixed latent birth with an explicit RoPE route 4
4.3 Component 2 — block-wise online sparse innovations 4
4.4 Component 3 — attention directly on latent plus sparse residual 5
4.5 Component 4 — local hot/cold softmax merge 5
4.6 Component 5 — bottleneck-aware joint budget and placement 5
4.7 Attention data flow (nothing large leaves a stage) 5

5 Theory targets **6**

6 Evaluation plan **6**
6.1 Project ramp 6
6.2 Testbed, baselines, metrics 6
6.3 Performance gates (targets, not promised results) 7

1 Why this idea, among the six

Six directions were considered. The earlier draft treated compressed-domain attention as the novel core. That is no longer defensible: Eigen Attention [12] already computes in a low-rank space, Palu [30] creates latent KV from factorized projections, and RAP [13] removes RoPE-induced reconstruction for aligned projections. The revised paper is a **new systems setting and co-design**: a RoPE-safe latent cache plus sparse prompt-specific innovations, jointly budgeted with heterogeneous pipeline placement to keep each layer and its cache stationary.

Candidate	Nov.	Feas.	Pub.	Verdict
1. Latent–residual Shufflerless-CI	Med–High	High	High	**Selected** (flagship)
2. Compute-at-the-cache (exact distributed softmax)	Low–Med	High	Med	Local hot/cold merge only
3. Predictive sparse KV prefetch	Low–Med	High	Med	Complementary; near ScoutAttention
4. Session-specialized weight residency	Med	Med	Med–High	Strong but separate (weights, not KV)
5. Risk-aware repartitioning	Low–Med	High	Med–High	*Merged in* as the no-shuffle planner
6. Compute/load/fetch/skip controller	Med	Med	Med	*Merged in* lightly as the adaptive controller

The selected project deliberately borrows its compression primitives and claims novelty at their *coupling*: (i) fixed RoPE-safe latent birth, (ii) sparse residuals created once when a hot block ages, and (iii) a stage planner that allocates rank and residual bytes where they reduce the pipeline bottleneck. Remote KV sharding and reinforcement learning are removed from the first paper because they add network round trips and scope without helping the main performance claim.

2 Motivation

Motivation in one paragraph

On-device and edge LLM decoding hits the **memory wall**: model weights plus a KV cache that *grows every token* cannot all stay in the memory budget assigned to an accelerator stage. Reactive paging [23] then creates transfers, synchronization, and tail latency. Pure activation-side compression is not the answer: running an SVD as the cache grows can cost more than the attention it saves. The research question is therefore: *can the KV be born small, corrected cheaply for each prompt, and co-placed with its owning layers so that steady-state decoding has no large cache transfers or reconstruction buffers?*

2.1 The three failure modes of compression-alone

Any scheme that merely stores KV compactly can still incur:

1. **Load:** the compressed data is fetched from CPU when needed;
2. **Reconstruct:** the dense KV tensor is rebuilt on the GPU before attention;
3. **Buffer bloat:** temporary reconstruction buffers consume the saved memory.

Shufflerless-CI removes all three by (i) generating the low-rank bulk through fixed projection factors, not runtime decomposition; (ii) adding only a sparse block-wise residual when an exact

hot block ages; and (iii) keeping every layer and its KV on one owner stage. On Jetson's unified memory, "shuffle" includes explicit copies, page migration, and synchronization; all must be measured rather than inferred from allocation APIs.

3 Background and related work

3.1 The FSU pipelined-inference poster (the base system)

The poster's **CI-Inf** [1] partitions a decoder by layers across heterogeneous Jetson devices; each device holds whole layers and keeps its layers' KV cache resident, avoiding CPU↔GPU swapping and cutting per-token latency 40% (325→192 ms). Shufflerless-CI extends CI-Inf so that residency is *guaranteed up to a budgeted context length*, via a latent–residual cache and joint stage placement. Placement/parallelism machinery is inherited from HexGen [2] and Jupiter [3]; fairness across devices from FOCI [4].

3.2 KV-cache compression — and the novelty boundary

Low-rank, sparse, and hybrid KV compression are active research, so none is by itself novel:

- **ShadowKV** [7]: low-rank keys kept on GPU, values *offloaded to CPU* and loaded back on demand.
- **Lexico** [8]: sparse coding over a universal dictionary; KV is *reconstructed* from atoms before use.
- **GEAR** [9]: quantization + a low-rank term for quantization error + a sparse term for outliers — literally low-rank+sparse, but reconstructed.
- **RocketKV** [10]: two-stage eviction + hybrid top-k sparse attention (single device).
- **Eigen Attention** [12]: performs attention directly in a low-rank space; **RAP** [13] uses RoPE-aligned pruning to avoid reconstruction.
- **OjaKV** [14]: combines exact anchors with an online-adapted low-rank subspace; **STAR-KV** [15] performs fine-grained adaptive rank control; and **LCA** [16] condenses context directly inside an MLA latent space.

The defensible gap is narrower: existing cache methods optimize a single model/device cache; CI-Inf fixes the layer split. This project jointly chooses the RoPE-safe projection, sparse innovation budget, exact hot window, and heterogeneous layer partition under an explicit no-shuffle constraint. This is a *novelty hypothesis*, not a settled fact; the literature audit must be refreshed immediately before submission.

3.3 Collaborative / distributed attention (the aggregation primitive)

The exact-merge machinery Shufflerless-CI uses is grounded in **online-softmax** computation [19] and **FlashAttention** [20], which compute attention block by block via running max, sum, and output statistics. **Ring Attention** [21] distributes those blocks across devices but moves KV blocks. **CAKE** [11] chooses whether to compute or load KV; **ScoutAttention** [22] performs GPU–CPU collaborative block-sparse attention. Shufflerless-CI instead treats remote per-layer attention as an emergency fallback: the fast path keeps the cache local to its layer owner and uses online-softmax only to merge local exact-hot and compressed-old blocks.

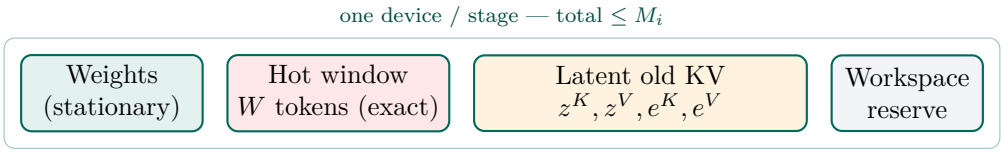
3.4 Tensor foundations

The low-rank+sparse model is Robust PCA [24]; factorizations are CP/Tucker [25] and Tensor-Train [26]; low-rank+sparse weight structure is LoSparse [27]; attention low-rankness is Linformer [28]; heavy-hitter token sparsity is H2O [29]; low-rank KV projection precedents include Palu [30], Eigen Attention [12], and MLA [18].

4 Method

4.1 The stationarity principle and per-stage memory layout

Each pipeline stage *permanently* stores its weights, an exact GPU-resident window of the W most recent KV entries, a compressed representation of older KV, and enough workspace that decoding never triggers emergency swapping:



4.2 Component 1 — fixed latent birth with an explicit RoPE route

For each layer/head, factorize the pretrained key/value projections *offline* using a small calibration set. A token hidden state x_t produces latent cache entries

$$z_t^K = x_t U_K \in \mathbb{R}^{r_K}, \qquad z_t^V = x_t U_V \in \mathbb{R}^{r_V}.$$

There is **no runtime SVD**. For RoPE models, an arbitrary hidden-dimension projection cannot simply be absorbed because the position-dependent rotation follows the key projection. The implementation must therefore choose one of two explicit, measurable routes:

1. **plug-in route:** preserve complete RoPE coordinate pairs, following the RoPE-aligned idea in RAP [13]; this is the low-risk first prototype;
2. **higher-compression route:** cache a small exact positional key k_t^R and a separate semantic latent, following MLA/MHA2MLA [18, 17]; use calibration plus parameter-efficient tuning rather than full pretraining.

Both routes make the KV *born compressed*, which is LoRA-like in timing, although the factors serve inference activations rather than representing a trainable weight update.

4.3 Component 2 — block-wise online sparse innovations

The fixed subspace cannot capture every prompt. Keep the newest W tokens exact. When a block B ages out, form its reconstruction residual once,

$$E_B^K = K_B - \hat{K}_B(z_B^K), \qquad E_B^V = V_B - \hat{V}_B(z_B^V),$$

retain only the top-$k_{\ell h}$ blocks/entries (with indices and low-bit values), and then discard the dense block. This is the only input-dependent compression step. It is amortized over a block and never decomposes the accumulated cache. OjaKV [14] adapts the subspace online; this design instead keeps the RoPE-safe basis fixed and spends online work only on sparse innovations, avoiding basis-update and synchronization cost.

4.4 Component 3 — attention directly on latent plus sparse residual

For an old block, scores take the form

$$s_{tB} \;\approx\; q_t^C (z_B^K)^\top + \langle R_t q_t^R, R_B k_B^R \rangle + q_t (E_B^K)^\top,$$

where the positional term is omitted for the RoPE-aligned plug-in route because it is already handled by its paired coordinates. The value result is evaluated as

$$p_B V_B \;\approx\; (p_B z_B^V) P_V^\top + p_B E_B^V,$$

with $P_V^\top$ fused into the output projection when the chosen parameterization permits. The old dense KV is never materialized. A fused block-sparse kernel is essential; unfused PyTorch serves only as a correctness prototype and is not a performance result.

4.5 Component 4 — local hot/cold softmax merge

The exact hot window and compressed old blocks remain on the *same stage*. Each block j produces online-softmax statistics

$$m_j = \max_t s_{jt}, \qquad \ell_j = \sum_t e^{\,s_{jt} - m_j}, \qquad o_j = \sum_t e^{\,s_{jt} - m_j} V_{jt}, \qquad s_j = q K_j^\top / \sqrt{d},$$

which merge exactly [19, 20] for the scores supplied by the exact and approximate blocks:

$$m = \max_j m_j, \qquad \ell = \sum_j e^{\,m_j - m} \ell_j, \qquad o = \sum_j e^{\,m_j - m} o_j, \qquad \text{output} = o/\ell.$$

This merge introduces no extra approximation. Remote KV sharding is not used on the fast path, because a network round trip at every attention layer would erase the expected gain.

4.6 Component 5 — bottleneck-aware joint budget and placement

For an admitted request with declared prompt length and `max_new_tokens`, the planner jointly chooses the contiguous layer partition π, exact window W_ℓ, projection ranks r_ℓ, sparse budgets k_ℓ, and bit-widths b_ℓ. It enforces, for every device i,

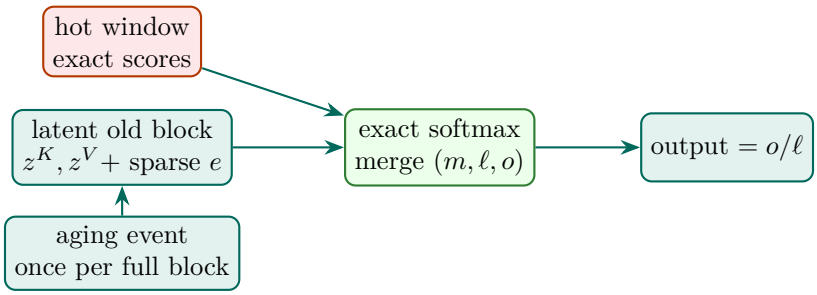
$$\underbrace{\text{weights}_i}_{\text{fixed}} + \underbrace{\text{hot KV}_i}_{W \text{ tokens}} + \underbrace{\text{compressed old KV}_i(r_\ell, k_\ell)}_{\text{grows slowly}} + \underbrace{\text{workspace}_i}_{\text{reserve}} \;\le\; M_i,$$

and minimizes the pipeline bottleneck rather than average work:

$$\min_{\pi, W, r, k, b} \; \max_i T_i(\pi, W, r, k, b) \quad \text{s.t.} \quad \sum_{\ell, h} \mathcal{E}_{\ell h}(r, k, b) \;\le\; \epsilon.$$

Here $\mathcal{E}_{\ell h}$ is a calibrated attention-output or logit-distortion proxy. The planner allocates each extra byte where it most reduces quality error *without slowing the maximal stage*. Admission control makes the no-shuffle guarantee deterministic for accepted requests; a chance-constrained oversubscription mode is optional future work, not part of the first-paper claim.

4.7 Attention data flow (nothing large leaves a stage)



5 Theory targets

Theory target

(T1) Exactness of the merge. The segmented softmax merge (Component 4) is *exact for the supplied block scores*: the only approximation is the latent+sparse KV representation, isolated to the compressed segment's scores s_{jt} and output o_j.

Theory target

(T2) Attention-relevant error bound. Bound the attention-output error $\|\hat{o}/\hat{\ell} - o/\ell\|$ in terms of (i) the residual KV energy after rank-r + k-sparse and (ii) the *missed attention mass*, rather than the raw reconstruction error $\|K - \hat{K}\|$. This is the useful result: it ties tolerable compression to what attention actually weights, so aggressive compression is safe exactly where attention mass is negligible.

Target Theorem 1 (no-shuffle guarantee)**.** *For every request admitted by Component 5 with prompt length plus* `max_new_tokens` *at most L, Shufflerless-CI performs no large KV transfer during steady-state decoding and the measured per-stage peak resident memory does not exceed M_i, provided the implementation respects the reserved workspace. This is a deterministic admission-control claim, not a probabilistic prediction claim.*

Theory target

(T3) Complexity crossover. Characterize the (r, k, W, B) regime in which Components 1–3, *including* amortized aging, index/bitmap traffic, RoPE handling, and kernel-launch overhead, are faster than dense attention or paging. This prevents memory compression alone from being reported as a latency win.

6 Evaluation plan

6.1 Project ramp

1. Calibrate fixed projection factors and measure residual concentration per layer/head; compare pre- and post-RoPE spectra, but do not run SVD during generation.
2. Implement the RAP-aligned plug-in route for one layer; add the decoupled-RoPE route only after the first route passes the quality and latency gates.
3. Add block-wise top-k residual extraction and a fused latent–residual attention kernel.
4. Measure the full crossover, including the once-per-block aging cost (T3).
5. Integrate one CI-Inf [1] stage, then jointly optimize the layer partition and layer/head budgets.
6. Enforce admission control and validate the no-shuffle claim with transfer/page-fault traces rather than allocator statistics alone.

6.2 Testbed, baselines, metrics

Testbed: the poster's 3× Jetson Orin Nano cluster [1], LLaMA-3.2-3B, 16/6/6 split, plus the partition selected by the new planner. **Baselines:** dense-KV CI-Inf; single-device and three-device dense inference under the same precision and power mode; Palu [30], Eigen Attention [12], RAP [13], OjaKV [14], and STAR-KV [15]; paging [23]; and ScoutAttention [22] where a fair

Jetson port is possible. Report unavailable kernels/code as a limitation rather than silently omitting the method. **Metrics:**

- explicit-copy bytes, migrated/page-faulted bytes, and synchronization time per token (the headline target is zero large KV movement for admitted requests);
- network bytes per token; maximum accelerator-resident memory;
- time-to-first-token and P50/P95 token latency; energy per token;
- maximum admitted context length before any shuffle, at batch sizes 1 and > 1;
- LongBench / RULER quality, and *GSM8K / coding accuracy* — because aggressive cache approximation disproportionately harms chained reasoning, this is the key quality guard.

6.3 Performance gates (targets, not promised results)

- **Representation gate:** at least 2× KV reduction with at most one percentage point average task degradation; otherwise stop before systems integration.
- **Kernel gate:** at least 1.25× attention speedup over dense attention at a memory-bound context, after amortizing aging. If it fails, make context capacity and tail latency—not mean token latency—the paper's primary result.
- **System target:** 1.5× end-to-end throughput or a 2–4× longer admitted context at matched quality, with lower P95 latency than paging. Short-context batch-1 decoding is a required negative control and may show no gain.

Main risk & mitigation

(a) RoPE handling can erase the low-rank speedup, as reported for Eigen Attention; make RAP-aligned projection the plug-in fallback and treat decoupled RoPE as an ablation. **(b)** Sparse residual indexing may be slower than dense math; use block sparsity and the kernel gate, falling back to latent-only or dense per layer. **(c)** The fixed subspace may fail under prompt shift; increase the innovation budget or keep sensitive heads exact rather than updating a basis online. **(d)** Jetson unified memory can make "CPU–GPU bytes" ambiguous; trace migrations, copies, faults, stalls, and memory-controller traffic separately.

Defensible paper claim

Shufflerless-CI is strongest as a *systems co-design*, not as a new decomposition: *RoPE-safe latent birth for the stable bulk, block-wise sparse innovations for input-specific error, and bottleneck-aware heterogeneous placement with deterministic no-shuffle admission.* This version removes per-token SVD, remote per-layer KV calls, and an RL controller from the critical path—the three choices most likely to erase performance.

Reviewer-style verdict

Accept with revisions. Faster has the highest ceiling (8/10) because the mechanism removes cache transfer, dense reconstruction, and unnecessary online decomposition; Cheaper is 8/10 through longer contexts on fixed hardware; Stronger is 6/10 because sparse innovations address prompt shift. Higher is 5/10 (quality preservation, not improvement) and Broader is 5/10. The remaining major flaw is novelty proximity to Eigen Attention, RAP, OjaKV, LCA, and 2026 hybrid KV methods. The defense is a decisive joint-optimization ablation: fixed partition plus the same compressor versus joint representation/placement, with measured end-to-end gains. Feasibility is yellow until the one-layer fused kernel passes the representation and kernel gates.

References

[1] H. Zhou and X. Zhang. *Poster: Distributed Pipelined Collaborative Inference for LLMs on Edge Device Clusters.* ACM HotMobile 2026.

[2] Y. Jiang et al. *HexGen: Generative Inference of Large Language Model over Heterogeneous Environment.* ICML 2024. arXiv:2311.11514.

[3] S. Ye et al. *Jupiter: Fast and Resource-Efficient Collaborative Inference of Generative LLMs on Edge Devices.* IEEE INFOCOM 2025. arXiv:2504.08242.

[4] H. Zhou and X. Zhang. *Fairness-Oracular MARL with Competitor-Aware Signals for Collaborative Inference.* NeurIPS 2025 Workshop (AI4NextG).

[5] L. Zu, H. Zhou, X. Zhang. *Behavior-Adaptive Q-Learning: A Unifying Framework for Offline-to-Online RL.* arXiv:2511.03695, 2025.

[6] L. Zu, Y. Qian, S. Chakraborty, X. Zhang. *From Static Constraints to Dynamic Adaptation: Sample-Level Constraint Relaxation for Offline-to-Online RL (DARE).* ICML 2026. arXiv:2511.03828.

[7] H. Sun et al. *ShadowKV: KV Cache in Shadows for High-Throughput Long-Context LLM Inference.* ICML 2025 (PMLR v267). arXiv:2410.21465.

[8] J. Kim, J. Park, J. Cho, D. Papailiopoulos. *Lexico: Extreme KV Cache Compression via Sparse Coding over Universal Dictionaries.* ICML 2025 (PMLR v267). arXiv:2412.08890.

[9] H. Kang, Q. Zhang, G. Jeong, T. Krishna, T. Zhao. *GEAR: An Efficient Error Reduction Framework for KV Cache Compression in LLM Inference.* PMLR v262, 2024. arXiv:2403.05527.

[10] P. Behnam et al. *RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression.* ICML 2025 (PMLR v267). arXiv:2502.14051.

[11] S. Jin et al. *Compute Or Load KV Cache? Why Not Both? (CAKE).* ICML 2025 (PMLR v267). arXiv:2410.03065.

[12] U. Saxena, G. Saha, S. Choudhary, K. Roy. *Eigen Attention: Attention in Low-Rank Space for KV Cache Compression.* Findings of EMNLP 2024.

[13] J. Xin et al. *RAP: KV-Cache Compression via RoPE-Aligned Pruning.* arXiv:2602.02599, 2026.

[14] Y. Zhu et al. *OjaKV: Context-Aware Online Low-Rank KV Cache Compression.* Findings of ACL 2026.

[15] P. Bhatnagar et al. *STAR-KV: Low-Rank KV Cache Compression via Soft Thresholding for Adaptive Rank Control.* arXiv:2606.08382, 2026.

[16] *Latent-Condensed Transformer for Efficient Long Context Modeling.* ACL 2026.

[17] Y. Ji et al. *Towards Economical Inference: Enabling DeepSeek's Multi-Head Latent Attention in Any Transformer-based LLMs.* ACL 2025.

[18] DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434, 2024.

[19] M. Milakov, N. Gimelshein. *Online Normalizer Calculation for Softmax.* arXiv:1805.02867, 2018.

[20] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* NeurIPS 2022. arXiv:2205.14135.

[21] H. Liu, M. Zaharia, P. Abbeel. *Ring Attention with Blockwise Transformers for Near-Infinite Context.* NeurIPS 2023. arXiv:2310.01889.

[22] *ScoutAttention: Efficient KV Cache Offloading via Layer-Ahead CPU Pre-computation for LLM Inference.* DAC 2026. arXiv:2603.27138.

[23] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM).* SOSP 2023. arXiv:2309.06180.

[24] E. J. Candès, X. Li, Y. Ma, J. Wright. *Robust Principal Component Analysis?* J. ACM 58(3):11, 2011.

[25] T. G. Kolda, B. W. Bader. *Tensor Decompositions and Applications.* SIAM Review 51(3):455–500, 2009.

[26] I. V. Oseledets. *Tensor-Train Decomposition.* SIAM J. Sci. Comput. 33(5):2295–2317, 2011.

[27] Y. Li et al. *LoSparse: Structured Compression of Large Language Models based on Low-Rank and Sparse Approximation.* ICML 2023. arXiv:2306.11222.

[28] S. Wang, B. Z. Li, M. Khabsa, H. Fang, H. Ma. *Linformer: Self-Attention with Linear Complexity.* arXiv:2006.04768, 2020.

[29] Z. Zhang et al. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models.* NeurIPS 2023. arXiv:2306.14048.

[30] J. Chang et al. *Palu: Compressing KV-Cache with Low-Rank Projection.* ICLR 2025. arXiv:2407.21118.